

4 章	パッケージのメタデータ	37
4.1	依存関係：パッケージには何が必要か	38
4.1.1	バージョン管理	40
4.1.2	その他の依存関係	40
4.2	Title と Description：パッケージは何をするのか.....	41
4.3	作成者：開発したのは誰か	43
4.3.1	CRAN では	44
4.4	ライセンス：パッケージを使えるのは誰か.....	44
4.4.1	CRAN では	46
4.5	バージョン	46
4.6	その他のコンポーネント	47

dev 113 branches 13 tags

Go to file

Add file

<> Code

mimitsuyo Merge pull request #801 from ichimomo/new_graph 731227c 5 days ago 1,743 commits

.github	Update pkgdown.yaml
R	Merge pull request #801 from ichimomo/new_graph
data-raw	fix test
data	fix test
inst	add setting_release argument in mak_future_data
man	update bubble plot manual
tests	add box.padding option for plot_SR
tools	remove_future_vignette
vignettes	fix code
.Rbuildignore	Add pkgdown files
.gitignore	fix
.travis.yml	Wait Travis, wait
DESCRIPTION	add scales to dependency pkg
LICENSE	Update LICENSE
NAMESPACE	Merge pull request #801 from ichimomo/new_graph
README.md	status budgeをgithubのものに変更

7 months ago

frasyr / DESCRIPTION

mimitsuyo add scales to dependency pkg

Code

Blame

46 lines (46 loc) · 1.47 KB

```

1 Package: frasyr
2 Title: Fisheries Research Agency (FRA) provides the method for calculati
3 Version: 2.2.0.3
4 Authors@R: c(person(given="Momoko", family="Ichinokawa", email = "ichimo
5               person(given="Akira", family="Hayashi", email = "akirahayashi
6               person(given="Mitsuyo", family="Miyagawa", email = "mmiyagawa
7               person(given="Shin", family="Fukui", email = "shinfukui@fra.a
8               person(given="Kohei", family="Hamabe", email = "hamabek@fra.a
9               person(given="Shota", family="Nishijima", email = "nishijim
10              person(given="Hiroshi", family="Okamura", email = "okamura@
11 Description: This package estimates stock status with stock assessment m
12 Depends: R (>= 3.5.0)
13 License: Apache Licence 2.0
14 Encoding: UTF-8
15 LazyData: true
16 RoxygenNote: 7.2.1
17 Imports:
18     ggplot2,
--

```

DESCRIPTIONファイルについて

- 役割：パッケージに関する重要なメタデータを保存

- ↳
 - 誰がこのパッケージを利用できるのか（ライセンス）
 - 問題がある場合の問い合わせ先

- 全てのパッケージに必要であり，パッケージの特徴を記述

↳ RstudioとdevtoolsはDESCRIPTIONファイルを含む任意のディレクトリをパッケージであるとみなす

- `devtools::create("自分の作りたいパッケージ名")`

→自動的に必要最小限のDESCRIPTIONファイルが追加される

```
DESCRIPTION
1 Package: calcmmy
2 Title: What the Package Does (One Line, Title Case)
3 Version: 0.0.0.9000
4 Authors@R:
5   person("First", "Last", , "first.last@example.com", role = c("aut", "cre"),
6     comment = c(ORCID = "YOUR-ORCID-ID"))
7 Description: What the package does (one paragraph).
8 License: `use_mit_license()`, `use_gpl3_license()` or friends to pick a
9   license
10 Encoding: UTF-8
11 Roxygen: list(markdown = TRUE)
12 RoxygenNote: 7.2.1
```

❏ 自分の例には，
Depends: と LazyData:
の部分は自動ではなかった

❏ Debian制御書式（DCF）
フィールド名： 値
※値が複数行にまたがる場合は，インデントする（通常スペース4つでインデント）

DESCRIPTIONの主要なフィールドの使い方

4.1. 依存関係：パッケージには何が必要か

- DESCRIPTIONの役割：パッケージを動作させるために必要なパッケージを一覧化
↳ Rには潜在的な依存関係を記述するための方法がたくさんある

例) Imports:

dplyr,
ggvis

❏ パッケージの動作に必要な場合はImports

- ここで記載されたパッケージがインストール済みでない場合はインストールされる
- しかし、library(xx)が呼ばれるわけではないことに注意
- Best practiceはpackage名::function名()という構文を使って、外部の関数を明示的に参照
→ (利点) どの関数が外部のパッケージに依存しているか簡単に特定
→ (欠点) 軽微なパフォーマンスの悪化 (5マイクロ秒のオーダー)

Suggests:

dplyr,
ggvis,

❏ パッケージの動作には必要ではないが、活用することができる場合はSuggests

- 自動的にはインストールされない
- パッケージを使う前に自分でそのパッケージが有効かを確認する必要あり
→ 例) if(!requireNamespace("パッケージ名", quietly=TRUE))
install.packages("パッケージ名")

DESCRIPTIONの主要なフィールドの使い方：Imports, Suggests

- Imports, SuggestsをDESCRIPTIONに追加する最も簡単な方法は

```
usethis::use_package("dplyr") # Default is "Imports"
#> ✔ Adding 'dplyr' to Imports field in DESCRIPTION
#> • Refer to functions with `dplyr::fun()`

usethis::use_package("ggplot2", "Suggests")
#> ✔ Adding 'ggplot2' to Suggests field in DESCRIPTION
#> • Use `requireNamespace("ggplot2", quietly = TRUE)` to test if package is installed
```

4.1.1 バージョン管理

- 特定のバージョンのパッケージに依存している場合は、パッケージ名の後に指定

```
Imports:
  dplyr (>= 1.0.0),
  tidyr (>= 1.1.0)
```

The `usethis::use_package()` convenience function also helps

```
# exact version
usethis::use_package("dplyr", min_version = "1.0.0")
```

- ❑ 多くの場合は、特定のバージョンを指定するよりも、**最小限のバージョンを指定**したほうがよい
- ❑ 一般的には、常にバージョンを指定したほうが良い
- ❑ 他の方法を知らない限りは、**自分が今使用しているバージョンと同じかそれ以上のバージョンを常に指定**

DESCRIPTIONの主要なフィールドの使い方: Depends, LinkingTo, Enhances

4.1.2 その他の依存関係(その他3つのフィールド)

- Depends : R2.1.4.0 (2011年) の名前空間の公開前は他のパッケージに依存する唯一の方法
現在は、**Depends**ではなく**Imports**を使うべき。それでもDependsを使うときの説明は8章。
→Dependsを使用して特定のバージョンのRを要求することもできる。
- LinkingTo : ここに記載されているパッケージは、他のパッケージの中のCやC++のコードに依存
(詳細は14.2.9 「コンパイル済みのコード」で)
- Enhances : ここに記載されているパッケージは、開発したパッケージによって「強化」されたものの (=他のパッケージで定義されたクラスのメソッドを提供していることを意味する) が。利用はお勧めしない

自分のパッケージがRの外部で必要なものは、SystemRequirementsフィールドに記載することができるが、これは単なるテキストフィールドで、自動的にチェックされない。

詳細なシステム要件 (およびそれらのインストール方法) は、**README**にも含める必要あり

```
SystemRequirements: C++17
SystemRequirements: GNU make
SystemRequirements: TensorFlow (https://www.tensorflow.org/)
```

DESCRIPTIONの主要なフィールドの使い方: Title, Description

4.2. TitleとDescription: パッケージは何をするのか

- **Title** : パッケージについて1行で説明したもの. パッケージの一覧表示によく使用
プレーンテキストで記載, ピリオドで終わらないように. 短く.
- **Description** : Titleより詳細に説明したもの. 複数文可だが1段落に制限. 説明が複数行になる場合,
各行は80文字より大きくなってはいけない.

Package: frasyr

Title: Fisheries Research Agency (FRA) provides the method for calculating sustainable yield (SY) with R

Description: This package estimates stock status with stock assessment models and calculating RPs and ABC for Japanese fisheries management.

Package: spict

Type: Package

Title: Stochastic surplus Production model in Continuous-Time (SPiCT)

Description: Fits a surplus production model to fisheries catch and biomass index data.

- 適切なタイトルと説明(Description)をつけることが大切. CRANにリリース予定なら特に重要
- Descriptionに記載できる量は決まっているので, README.mdを作成しておくことを推奨

DESCRIPTIONの主要なフィールドの使い方:Authors@R

4.3. Authors：開発したのは誰か

パッケージの作成者，連絡先など記述（最近はcommentにORCIDを書いたりも）

```
Authors@R: c(person(given="Martin Waever",
  family="Pedersen",
  email="wpsgodd@gmail.com",
  role=c("aut", "cre", "cph")),
  person(given="Casper Willestofte",
  family="Berg",
  email="cbe@aqua.dtu.dk",
  role=c("aut"),
  comment=c()),
  person(given="Tobias Karl",
  family="Mildenberger",
  email="tobm@aqua.dtu.dk",
  role=c("aut"),
  comment=c(ORCID="0000-0002-6631-7524")),
  person(given="Alexandros",
  family="Kokkalis",
  email="alko@aqua.dtu.dk",
  role=c("aut"),
  comment=c(ORCID="0000-0002-9662-6272")))
```

role は3文字のコードで指定する.重要な役割が4つ

- “aut” 著者
- “cre” 作成者(creator) または保守担当者，問題がある場合に問い合わせすべき人
- “ctb” 貢献者(contributor) パッチを提供するなど，小さな貢献をした人
- “cph” 著作権者(copy rights holder) 著作権が著者以外に保有されているときに使用
- “fnd” 資金提供者(funder)
 - ※このほかにも色々役割あてようと思えば色々ある

- 少なくとも1人のautとcreを含む必要あり
- URLやバグレポートの提出先も書くことできる

URL: <http://github.com/James-Thorson-NOAA/FishLife>

BugReports: <http://github.com/James-Thorson-NOAA/FishLife/issues>

当分の間使えるアドレスに，CRANのポリシーでは，1人あてでメーリングリストではないもの

DESCRIPTIONの主要なフィールドの使い方:License

4.4. License：パッケージを使えるのは誰か

- GPL-2, BSDなどといったオープンソースライセンスの標準的な略称を記載するか、より詳細な情報が記載されたファイルを参照するためにfile LICENSEと記載する可。
→この場合はLICENSEというファイルを作成して次のようなことを記載。

ライセンス例

Proprietary

独占所有権があります。組み込んだ製品の外部へ再配布してはいけません。

Do not distribute outside of Widgets Incorporated.

Permissive(寛容)なオープンライセンス

- MIT～BSD-2やBSD-3のライセンス条項に似てる。一つの制限（必ずライセンスとともにコードを配布）のもとに自由にコードを再配布可能。ライセンスファイルの作成必要（詳細は[MIT License | Choose a License](#)）
- Apache ～Frasyrはこれと思ったら、ライセンスファイルはGPL3のほうが。．．．ライセンスファイルの作成必要（詳細は[Apache License 2.0 | Choose a License](#)）

Copyleftなオープンライセンス(こっちのほうが厳しい)

- GPL ～コードをバンドルして配布する人は、必ずすべてのバンドルをGPL準拠の方法でライセンスしなければならない。これに加えて、変更を加えたバージョン（派生品）を配布する場合は、ソースコードも利用可能にしなければならない。GPL-3はGPL-2より厳しい（詳細は[GNU General Public License v3.0 | Choose a License](#)）
SPiCTやFishLifeはGPL-3だった。

CCO

コードとデータの全ての権利を放棄。よって、誰がどのような目的で使用しても構わない＝公有財産にする

DESCRIPTIONの主要なフィールドの使い方:Version

4.5. Version:

- 正式には、Rパッケージのバージョンは「. 」か「-」で区切られた整数を少なくとも2つ並べたもの
○ 1.0, 0.9.1-10 × 1, 1.0-devel
- バージョン番号は、後続のリリース時に上がるが、番号の変え方によってパッケージにどのような変更があったかを伝えることができる。
- リリースしたバージョンは、**3つの番号<major>.<minor>.<patch>で構成**する。
- 開発中のパッケージには、4つ目のコンポーネントがある。つまり、開発バージョン番号「. 9000」から始めるべき。例：パッケージの最初のバージョンは0.0.0.9000

ちなみに現在のFrasyrは2.2.0.3, FishLifeは3.0.0, SPiCTは1.3.7

4.6. その他のコンポーネント

- 他のフィールドについてはこの本の他の場所で説明
- 追加のメタデータを記述するために、独自のフィールドを作成することも可能。その際は、既存のフィールド名を使わない、正しい英単語を使うに注意

8 章

名前空間.....	93
8.1 動機付け	93
8.2 検索パス	94
8.3 名前空間	97
8.4 ワークフロー	98
8.5 エクスポート	99
8.5.1 S3	100
8.5.2 S4	101
8.5.3 RC	101
8.5.4 データ	101
8.6 インポート	101
8.6.1 R 関数.....	102
8.6.2 S3	102
8.6.3 S4	103
8.6.4 コンパイル済みの関数.....	103

dev 113 branches 13 tags

Go to file

Add file

<> Code

mmitsuyo Merge pull request #801 from ichimomo/new_graph 731227c 5 days ago

.github	Update pkgdown.yaml
R	Merge pull request #801 from ichimomo/new_graph
data-raw	fix test
data	fix test
inst	add setting_release argument in mak_future_data
man	update bubble plot manual
tests	add box.padding option for plot_SR
tools	remove_future_vignette
vignettes	fix code
.Rbuildignore	Add pkgdown files
.gitignore	fix
.travis.yml	Wait Travis, wait
DESCRIPTION	add scales to dependency pkg
LICENSE	Update LICENSE
NAMESPACE	Merge pull request #801 from ichimomo/new_graph
README.md	status budgeをgithubのものに変更

frasyr / NAMESPACE

 Merge pull request #801 from ichimomo/new_graph

Code

Blame

179 lines (178 loc) · 3.74 KB

```

1 # Generated by roxygen2: do not edit by hand
2
3 S3method(make_table,fit.SR)
4 S3method(make_table,fit.SRregime)
5 S3method(plot,future)
6 S3method(plot,futures)
7 S3method(print,myarray)
8 export(Generation.Time)
9 export(HCR_default)
10 export(SRF_BH)
11 export(SRF_HS)
12 export(SRF_RI)
13 export(SRplot_gg)
14 export(SRregime_plot)
15 export(apply_year_colum)
16 export(arrange_weight)
17 export(autocor.plot)

```

8章 名前空間

よく設計された名前空間は、パッケージの自己完結性を向上（他のパッケージと互いに干渉しない、実行環境に依存せずに機能することを保障）

8.1 動機付け

- 「名前」のための「空間」を定義

例) `nrow`
`#> function (x)`
`#> dim(x)[1L]`
`#> <bytecode: 0x1f681a0>`
`#> <environment: namespace:base>`



```
dim <- function(x) c(1, 1)
dim(mtcars)
#> [1] 1 1
nrow(mtcars)
#> [1] 32
```



新しくdimを定義しても、nrowが参照するのはnamespaceにあるbaseパッケージで定義されてるdim

- インポート：パッケージ内の関数から、他のパッケージの関数を検索する方法を定義
- エクスポート：開発したパッケージに含まれる関数のうち、外部に公開する関数を定義
(エクスポートしない関数はパッケージ内でのみ利用可能、最低限必要な関数のみエクスポート)
(エクスポートする関数が少ないほど、関数の衝突が起きる可能性も低くなるため)

8章 名前空間

8.2. 検索パス

- ある関数を呼び出すためには、Rは最初に対象の関数を検索する.
- Rはまずはグローバル環境から関数を探す.
- 見つからない時は、検索パスから検索.

↳ アタッチしている全てのパッケージのリスト (search())でリスト確認)

↳ 状態としてはlibrary()またはrequire()の処理されている

↓. 表: パッケージを使えるようにする4つの関数と、パッケージが見つからない場合に何が起こるか

	エラーが発生する	FALSE を返す
ロード	loadNamespace("x")	requireNamespace("x", quietly = TRUE)
アタッチ	library(x)	require(x, quietly = TRUE)

- パッケージをDESCRIPTIONファイルのDependsまたはImportsに列挙することで必要なときにインストールされることが保証される. **Imports**はロードするだけで、**Depends**はアタッチまで行う. 明確な理由がないかぎり、Importsで定義. 例外は、例えばanalogueパッケージはveganパッケージをベースに構築されていて、analogueはveganなしでは都合悪いときはDependsで定義
- **パッケージ内部ではrequire()やlibrary()は使用しない**

8章 名前空間

8.3. 名前空間

- 名前空間のディレクティブは10種類
そのうちの5つがエクスポート

`export()`

関数（S3 および S4 のジェネリックを含む）をエクスポートする

`exportPattern()`

パターンにマッチするすべての関数をエクスポートする

`exportClasses()` と `exportMethods()`

それぞれ S4 クラスとメソッドをエクスポートする

`S3method()`

S3 メソッドをエクスポートする

- 残りの5つがインポート

`import()`

対象のパッケージからすべての関数をインポートする

`importFrom()`

指定された関数（S4 ジェネリクスを含む）をインポートする

`importClassesFrom()` と `importMethodsFrom()`

それぞれ S4 クラスとメソッドをインポートする

`useDynLib()`

C ライブラリ（これについての詳細は「10章」：
ら関数をインポートする

これらのディレクティブを実際にタイプして作成するのは非推奨。代わりに **roxygen2** を使って **NAMESPACE** ファイルを生成

8章 名前空間

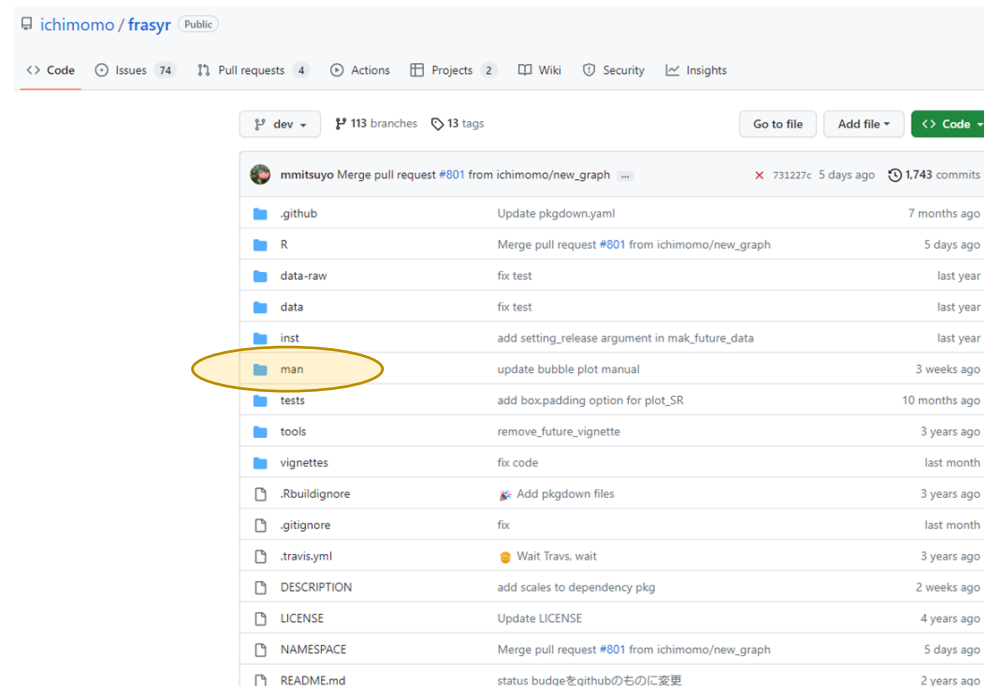
8.3. 名前空間

- Roxygen2を使う主なメリット3つ

1. 名前空間の定義が関連する機能のすぐ近くに記述されるため、インポート・エクスポート対象が分かりやすくなり可読性が向上
2. NAMESPACEファイルの詳細な定義を抽象化できる→覚える必要があるのは@exportタグのみ
3. NAMESPACEファイルを整理された状態に保つ

- Roxygen2はman/*.Rdファイルの生成も行う。

※ただし，関連したタグ(@exportなど)
を使用した場合のみ



8章 名前空間 (workflow, export)

8.4. ワークフロー

- 作成にはRoxygen2ブロック('#で始まる) とタグ (@で始まる) を使用
1. .Rファイルにroxygenコメントを追加する
 2. devtools::document()を実行して, roxygenコメントを.Rdファイルに変換する
 3. NAMESPACEファイルを確認し, テストを実行して正しい仕様になっているかを確認める
 4. 正しい機能がエクスポートされるまでこれを繰り返す

8.5. エクスポート

- @export をroxygenブロックに挿入する
- オブジェクトの種類に応じてexport(), exportMethods(),exportClass(),S3method()が生成される
- 他の人に使ってもらいたい機能をエクスポートする (エクスポートする対象はなるべく少なく)

```
#' @export
foo <- function(x, y, z) {
  ...
}
```

8章 名前空間 (export S3)

8.5.1 S3

- パッケージ外でS3クラスのインスタンス作成をできるようにしたい→@export
- S3methodのexportはかなり複雑

ケース1. exportしているジェネリックを実装したS3method → **全てのメソッドをエクスポート**
例) dplyrのcount.data.frame()というS3methodはgenericな関数であるdplyr::count()を実装

```
#' ... all the usual documentation for count() ...  
#' @export  
count <- function(x, ..., wt = NULL, sort = FALSE, name = NULL) {  
  UseMethod("count")  
}  
  
#' @export  
count.data.frame <- function(  
  x,  
  ...,  
  wt = NULL,  
  sort = FALSE,  
  name = NULL,  
  .drop = group_by_drop_default(x)) { ... }
```



```
...  
S3method(count,data.frame)  
...  
export(count)  
...
```

NAMESPACEファイルはこんな感じになる

この例は、count()をdata.frameという通常のclassにあてはめたが、では、独自のclassに当てはめたい場合は？

8章 名前空間 (export S3)

8.5.1 S3

ケース2. Genericな関数を独自のclassに当てはめて関数を作りたい場合は？

→genericなpackage(ここではdplyr)をDESCRIPTIONのImportsに書いて、これをnamespaceにimportして、S3methodをexportする。

```
Imports:  
...  
dplyr,  
...
```

```
#' @importFrom dplyr count  
#' @export  
count.tbl_lazy <- function(x, ..., wt = NULL, sort = FALSE, name = NULL) { ... }
```

```
S3method(count,tbl_lazy)  
...  
importFrom(dplyr,count)
```

DESCRIPTIONファイル

.Rファイル

NAMESPACE

ケース3. Suggestsに含まれるパッケージのgenericを実装したS3method

→suggestsに含まれるパッケージは参照できないので、パッケージの依存関係を設計し直す

ケース4. 内部で利用しているgenericを実装したS3method

→技術的な観点からはこれらのメソッドをエクスポートする必要はないが、S3methodをexportするのを推奨。Devtools::missing_s3()を使用すると、exportし忘れてるS3methodがリストアップ

8章 名前空間 (export S4)

8.5.2 S4

- パッケージ外でS4クラスを継承させたい場合→@export
- インスタンスの生成は許可したいが、クラスの継承はさせたくない場合→クラスでなくコンストラクタ関数で@export

```
#' @export  
setClass("A")
```

クラスの継承とnew("A",...)
によるインスタンス生成が
出来る

```
#' @export  
B <- setClass("B")
```

クラスの継承とnew("B",...)によ
るインスタンス生成が出来る。
自コード内ではB()という書き方
でインスタンスを生成できるが、
外部からはできない

```
#' @export C  
C <- setClass("C")
```

コンストラクタでの
new("C",...)とC(...)によるイ
ンスタンス生成は出来るが、
継承はできない

```
#' @export D  
#' @export setClass D  
D <- setClass("D")
```

継承と、new("D",...),D(...)
によるインスタンス生成可

- S4のgenericを公開して使えるようにしたい場合→@export
- S4のmethodを公開して使えるようにしたい場合→自分が定義していないgenericを実装したmethodだけを@exportすればよいはずだが、筆者は全てのmethodを@exportするのを推奨

8章 名前空間 (import)

8.5.3. RC

- S4クラスと同じ原則が適用
- 現在の実装では、パッケージ外からクラスを継承させることは通常不可能

8.6.インポート

- 自分のパッケージから::なしで呼び出したい外部関数の制御
- DESCRIPTIONにおけるImportsは、指定したパッケージがインストールされていることを保障しているだけで、利用可能な状態にしているわけではない
- 筆者は、パッケージをpkg::fun()のように参照する方法を推奨.
- NAMESPACEに記載するパッケージはImportsまたはDependsフィールドにも定義する必要あり

8章 名前空間 (import)

8.6.1 R関数

- 他のパッケージの関数を少しだけ使っているようなときに推奨するのは、**パッケージ名をDESCRIPTIONのImportsフィールドに記載しておき、その関数を明示的に::を使って呼び出す**
- 同じ関数を何度も使用する場合→**@importFrom pkg fun**でインポートすれば、::を使わずに済む (::を使用した呼び出しは、およそ5マイクロ秒ほど関数の評価時間を増やす)
- 他のパッケージの多くの関数を何度も使用する場合は、**@import package**を使ってそれらすべてをインポートすることも可能 (しかし、これはあらゆる場合に勧められる方法ではない。コードが読みにくくなる上に、関数名の衝突の可能性が高まる)

8.6.2 S3

- genericをimportすれば全てのmethodが利用できる

8章 名前空間 (import S4)

8.6.3 S4

- 別のパッケージで定義されているクラスを使うには、importしたクラスを継承するクラスの付近、あるいはimportしたクラスのgenericを実装するmethodの付近に **@importClassesFrom package ClassA ClassB...** のように定義
- 別のパッケージで定義されたgenericを使うには、importしたgenericを使用するmethodの付近に **@importMethodsFrom package GenericA GenericB...** のように定義
- S4はmethodsパッケージに実装されているので、このパッケージが利用できるかどうか確認する必要あり。 **DESCRIPTIONファイルにImports::methodsを記載**
- methodsパッケージはよく使うので、次のようにしてパッケージ全体をインポート
#' @imports methods
NULL
- あるいは、よく使う関数だけをimport
#' @importFrom methods setClass setGeneric setMethod setRefClass
NULL

9 章	外部データ	105
9.1	エクスポートされたデータ	105
9.1.1	データセットのドキュメント	107
9.2	内部データ	108
9.3	生データ	108
9.4	その他のデータ	109
9.5	CRAN に関する補足	109

9章 外部データ

- パッケージにデータを含めると便利→使用例を提供できる
- パッケージにデータを含める主な方法3つ
 1. バイナリデータを保存して，ユーザが使用できるようにする
→データをdata/に置く．使用例のためのデータセットはここに置くのがベスト
 2. 読み込まれたデータを保存したいけど，ユーザには使用させたくない
→データをR/sysdata.rdaに置く．関数に必要なデータなどはここに置くのがベスト
 3. 生データを保存したい場合
→データをinst/extdataに置く（frasyrのテストで使うデータなんかはここ）

9章 外部データ

9.1 エクスポートされたデータ

- パッケージデータを置く最も一般的な場所は **data/**
- このdirectory内の各ファイルは、（ファイル名と同じ名前の）1つのオブジェクトを `save()`によって保存した **.RDataファイル**でなければならない。
- ~~`devtools::use_data()`~~ を使用すれば、このルールに従ったデータを簡単に作成可能

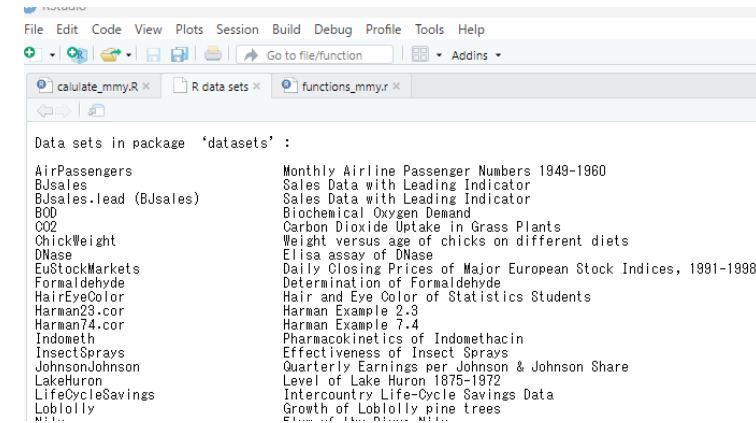
↳ **`usethis::use_data()`**

```
my_pkg_data <- sample(1000)
usethis::use_data(my_pkg_data)
```

- `data()`とするとRで使用できるデータセットの一覧出てくる
- 容量の大きな **データ** → **圧縮** の設定をいくつか変えて試す

```
usethis::use_data(compress =)
```

“bzip2”, “gzip”, “xz” など



9章 外部データ

9.1 エクスポートされたデータ

- **DESCRIPTION**に**LazyData: true**の記述があると、**データセットは遅延ロード**される
→つまり、使用されるまでデータはメモリを占有しない
- data/にある.rdaファイルを読み込むなら、LazyData:trueとしておいたほうが良い
↳use_data()を使って.rdaを作成する場合は、自動的にDESCRIPTIONに書かれる
- data/に置くデータは、生データをきれいに整理したデータであることが多い
↳その場合、データ整理に使用したコードをdata-raw/に置いておく。
→バンドルパッケージでは必要ないので、**.Rbuildignore**の**リストに加えておく**
→**usethis::use_data_raw()**とすれば、data-raw/フォルダーが作成、.Rbuildignoreも

```
usethis::use_data_raw()
```

```
usethis::use_data_raw("my_pkg_data")
```

9章 外部データ

9.1.1 データセットのドキュメント

- data/内のオブジェクトは常にエクスポートされる→データのドキュメントが必要
- 基本的には関数のドキュメントと同様だが，minorな違い
→データのドキュメントを直接作るのではなく，データセットの名前に対して作成し，R/にsaveする.

例) tidyrの中のwhoデータのドキュメントはR/data.Rにセーブ

```
#' World Health Organization TB data
#'
#' A subset of data from the World Health Organization Global Tuberculosis
#' Report ...
#'
#' @format ## `who`
#' A data frame with 7,240 rows and 60 columns:
#' \describe{
#'   \item{country}{Country name}
#'   \item{iso2, iso3}{2 & 3 letter ISO country codes}
#'   \item{year}{Year}
#'   ...
#' }
#' @source <https://www.who.int/teams/global-tuberculosis-programme/data>
"who"
```

@format データの概要を説明。データフレームの場合には、各変数を説明する定義リストを含む。変数の単位もここに記述しておく

@source データの由来の詳細を記述

注意：データセットには、名前空間の@exportは使用しない



Where is this in Frasyr?

9章 外部データ

9.2 内部データ

- 計算済みのデータテーブルを必要とする場合，data/に置くと，データがユーザにも見えてしまう．代わりにR/sysdata.rdaに保存
- R/sysdata.rdaを作成する最も簡易な方法は `usethis::use_data(x, internal = TRUE)`
- データの再現性のために，データ生成に使ったコードをパッケージのdata-raw/へ
- R/sysdata.rda内のオブジェクトはexportされないので，ドキュメントの必要はない．

9.3 生データ

- データのファイルをinst/extdataに置く
- Inst/extdata内のファイルパスを参照するためには`system.file()`を使用

```
system.file("extdata", package = "readxl") |> list.files()
#> [1] "clippy.xls"      "clippy.xlsx"    "datasets.xls"   "datasets.xlsx"
#> [5] "deaths.xls"     "deaths.xlsx"    "geometry.xls"   "geometry.xlsx"
#> [9] "type-me.xls"    "type-me.xlsx"

system.file("extdata", "clippy.xlsx", package = "readxl")
#> [1] "/home/runner/work/_temp/Library/readxl/extdata/clippy.xlsx"
```

ファイルが存在しないときは空の文字列が返される

9章 外部データ

9.4 その他のデータ

- テストのためのデータ～test directoryに直接容量の小さなファイルを置くことも可能
- Vignetteのためのデータ～すでに読み込まれているデータセットの動作を説明したい場合には，データをdata/に置く．生データをロードする方法を説明したい場合には，データをinst/extdataに置く．

9.5 CRANに関する補足

- パッケージのデータは1MBよりも小さくすべし．
- それより大きい場合には免除を受ける必要あり．
- データが最適に圧縮されているかを確認
 - `tools::checkRdaFiles()`を実行して，各ファイルに最適な圧縮方法を決める
 - `r usethis::use_data(compress =)`に最適な値を設定